

Assignment 2

- 📄 **Worth:** 5%
- 📅 **Due:** October 27, 2019 @ 23:59
- 🚫 **Penalty:** Late submissions lose 10% per day to a maximum of 3 days. Nothing is accepted after 3 days and a grade of 0% will be given.
- ✅ **Submission:**
 - ⚠️ Verify that your functions are named **EXACTLY** as specified in the questions, take the **EXACT INPUT**, and return the **EXACT OUTPUT**. This is because I will be using an automated test suite to run your assignment and if your functions are not created exactly as I've outlined them, **they will not pass the tests.**
 - Submit `Assignment-2-FIRSTNAME-LASTNAME.js` to the corresponding Moodle link.

Setup

1. Create a new folder in your `420-426-Multimedia-and-Internet` repo called `Assignment-2` using the `Finder`.
2. Open this new folder inside `VS Code`:
 - Create `Assignment-2-FIRSTNAME-LASTNAME.js`.
 - Create `index.html`.
 - Link the HTML to the JS with a `<script>` tag.
 - Open `index.html` in Chrome.
 - Open the Chrome `DevTools` by right-clicking on the page and hitting `Inspect`.
 - Open the `Console` tab.
3. You can test your code by either:
 - Calling your functions from the `console`, or
 - Calling your functions from `main.js` and `console.log()` the results.

As you complete each problem, make sure to **commit as you go**! This will keep track of all your changes as you code. What you do **not** want to do is complete the exercise without committing, and do one giant commit at the end.

1. ✕ Tic-Tac-Toe Referee ○

Tic-Tac-Toe, sometimes also known as X's and O's, is a game for two players (X and O) who take turns marking the spaces in a 3x3 grid. The player who succeeds in placing three respective marks in a *horizontal*, *vertical*, or *diagonal* rows (NW-SE and NE-SW) wins the game.

But we will not be playing this game. You will be the referee for this games results. You are given a result of a game and you must determine if the game ends in a win or a draw as well as who will be the winner.

Make sure to return `"X"` if the X-player wins and `"O"` if the O-player wins. If the game is a draw, return `"D"`.

Input: A game result as an array of 3 strings.

Output: "X" , "O" , or "D" as a string.

Example:

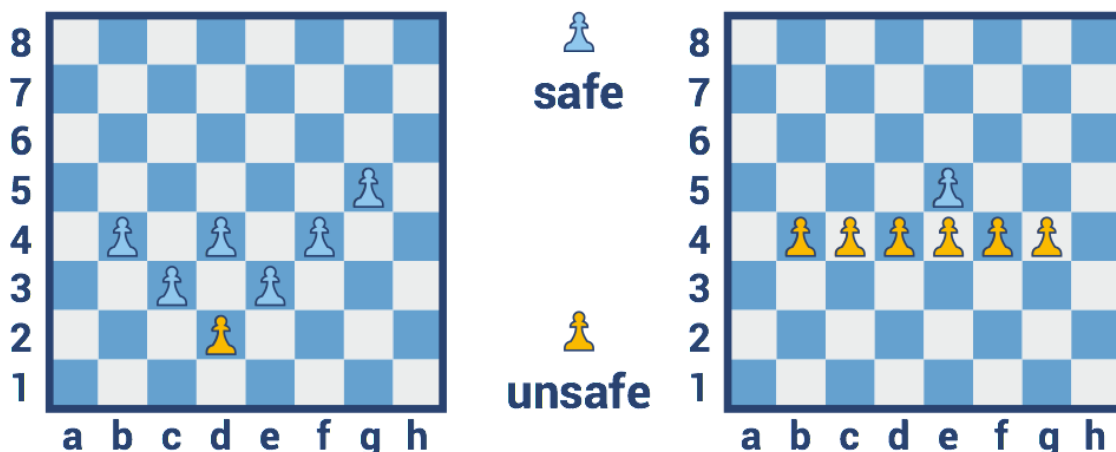
```
1  xoReferee([
2      ". .0",
3      "XXX",
4      "X00" ]) == "X"
5
6  xoReferee([
7      "00.",
8      "X0X",
9      "XX0" ]) == "O"
10
11 xoReferee([
12     "00X",
13     "XX0",
14     "0XX" ]) == "D"
```

2. ♟ Pawn Brotherhood ♟

Background: Chess is a two-player strategy game played on a checkered game board laid out in eight rows (called ranks and denoted with numbers 1 to 8) and eight columns (called files and denoted with letters a to h) of squares. Each square of the chessboard is identified by a unique coordinate pair — a letter and a number (ex, "a1" , "h8" , "d6"). For this mission we only need to concern ourselves with pawns. A pawn may capture an opponent's piece on a square diagonally in front of it on an adjacent file, by moving to that square. For white pawns, "in front" means ranks that are greater than the rank the pawn is currently on.

A pawn is generally a weak unit, but we have 8 of them which we can use to build a pawn defense wall. With this strategy, one pawn defends the others. A pawn is safe if another pawn can capture a unit on that square. We have several white pawns on the chess board and only white pawns.

Objective: You are given a set of square coordinates where we have placed white pawns. You should count how many pawns are safe.



Note: All of the pawns in the above image are on the **same team** (i.e. white). The blue and yellow is just to

denote which white pawns are safe and which are not.

Input: Placed pawns coordinates as an array of strings.

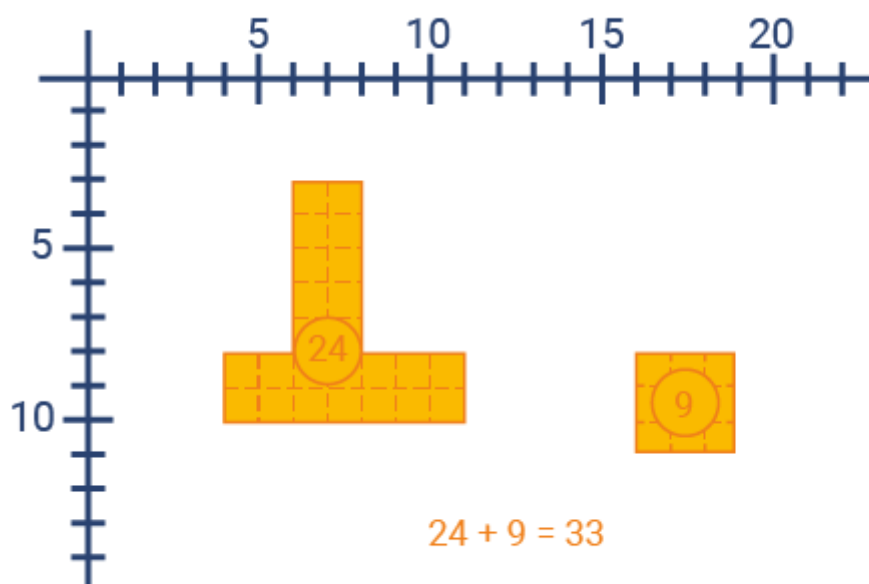
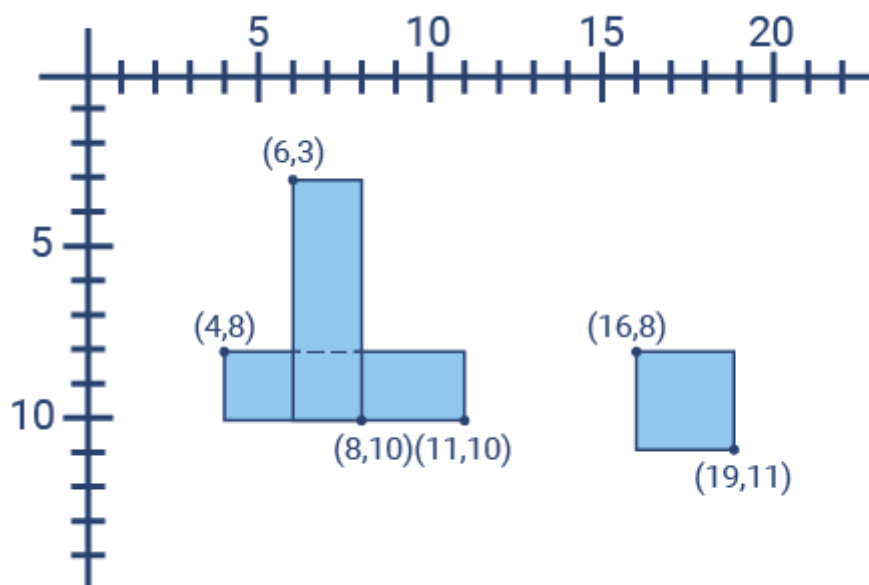
Output: The number of safe pawns as a integer.

Example:

```
1 safePawns(["b4", "d4", "f4", "c3", "e3", "g5", "d2"]) == 6
2 safePawns(["b4", "c4", "d4", "e4", "f4", "g4", "e5"]) == 1
```

3. Rectangle Union

Objective: Calculate the area covered by a union of rectangles. The rectangles can have a non-empty intersection, which means that a simple sum of given rectangle areas doesn't work. Every rectangle is represented as 4 integers. The first two integers are the coordinates of a left-top corner, and the next two - of a bottom right corner.



Note: No more than **TWO** rectangles can be intersecting at any given location. Meaning, we cannot have a "polygon" made up of 3+ rectangles.

Input: Array of arrays.

Output: Integer.

Example:

```
1  rectanglesUnion([
2      [6, 3, 8, 10],
3      [4, 8, 11, 10],
4      [16, 8, 19, 11]
5  ]) == 33
6
7  rectanglesUnion([
8      [16, 8, 19, 11]
9  ]) == 9
10
11 rectanglesUnion([
12     [16, 8, 19, 11],
13     [16, 8, 19, 11]
14 ]) == 9
15
16 rectanglesUnion([
17     [16, 8, 16, 8]
18 ]) == 0
19
20 rectanglesUnion([]) == 0
```

4. 🚂 Fast Train 🚉

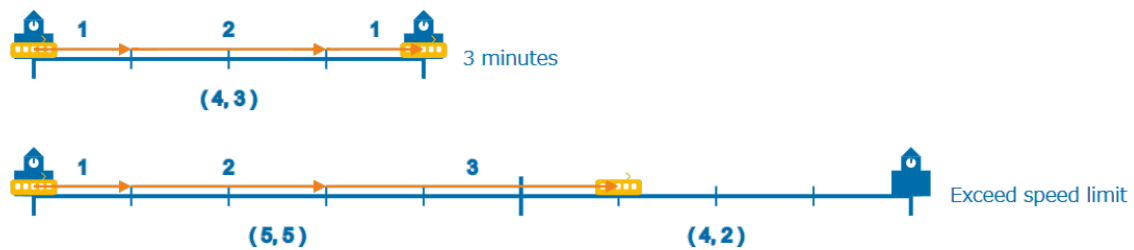
Objective: You are planning the train schedule and you want to know the minimum time between the stations.

Each section of the rail between stations is given in the input array. Each section is a tuple of distance and speed limit (both are integers). You can change the speed (**+1** , **-1** , and **±0**) at the start and every minute after that. The train runs by the same amount as the speed value in a minute.

Note: This means that a train with a speed of 2 will travel a distance of 2 before another minute passes and its speed can be changed again.

- Starting speed is 0.
- Speed limit is set for each section of the rail. You don't exceed it.
- You must reach the target station at speed 1 (because it's necessary to stop at the station).

You should return the minimum time (minutes) as an integer.



Input: Array of the section of the rail. Each section is an array of distance (as an integer) and speed limit (as an integer).

Output: The minimum time (minutes) as an integer.

Example:

```
1 fastTrain([[4, 3]]) == 3 // (1, 2, 1)
2 fastTrain([[9, 5]]) == 5 // (1, 2, 3, 2, 1)
3 fastTrain([[5, 5], [4, 2]]) == 6 // e.g. (1, 2, 2, 2, 1, 1)
```

Last Steps

Ensure that your JS file only contains 4 functions, and that they are all titled the exact same way as in the examples for each question. Yes, case matters.

🤔 What if I have helper functions that my "core" 4 functions call upon?

You can move these helper functions inside of your core functions. Your core functions can still call the helper functions because the helper functions will be inside the **scope** of the core function.

Example:

You can turn this:

```
1 function coreFunction() {
2     // ... code ...
3     helperFunction(); // Call the helper.
4 }
5
6 function helperFunction() {
7     // ... code ...
8 }
```

into this:

```

1  function coreFunction() {
2      // ... code ...
3      helperFunction(); // Call the helper.
4
5      function helperFunction() {
6          // ... code ...
7      }
8  }

```

Again, your final JS file should **only have 4 "core" functions!** Each one being: `xoReferee` , `safePawns` , `rectanglesUnion` , and `fastTrain` .

After you're done coding all 4 functions, **append** the following to the **end** of your `.js` file:

```

1  module.exports = {
2      xoReferee: xoReferee,
3      safePawns: safePawns,
4      rectanglesUnion: rectanglesUnion,
5      fastTrain: fastTrain
6  };

```

I'll need this in order to test your code.

 **If this is not present at the bottom of your code and/or written incorrectly, you will be penalized!** 

Basically, your code should look like this at the end:

```

1  function xoReferee(...) {...}
2  function safePawns(...) {...}
3  function rectanglesUnion(...) {...}
4  function fastTrain(...) {...}
5
6  module.exports = {
7      xoReferee: xoReferee,
8      safePawns: safePawns,
9      rectanglesUnion: rectanglesUnion,
10     fastTrain: fastTrain
11 };

```